Test Automation Proof of Architecture (PoA) — 4-Week Execution Playbook

Role: Senior Automation Architect | **Duration:** 4 weeks | **Scope:** 9 applications (Desktop, Web, Shell scripting) **Deliverables:** Application Assessment Report • Pilot Automation Scripts • AI-Native Testing Approach • Tool Recommendation • Scaled Automation Roadmap

---

1. Engagement Operating Model

1.1 Objectives

1. Assess automation feasibility and readiness of all 9 applications.
2. Identify and shortlist challenging test scenarios (beyond what the client already flags).
3. Pilot 2 shortlisted tools against those scenarios and capture objective evidence.
4. Recommend the best-fit enterprise tool stack with an AI-native testing approach.
5. Deliver a roadmap for scaled automation adoption.

1.2 Entry Criteria (secure BEFORE Day 1 — this is where 4-week PoAs die)

#	Item	Owner	Needed by
1	Access to all 9 apps (test environments, not prod)	Client IT	Day 1
2	Test user accounts with representative roles	Client IT	Day 1
3	Trial/eval licenses for candidate tools	You + Vendor	Day 3
4	VM/workstation with admin rights for tool installs	Client IT	Day 1
5	App SMEs identified (1 per app, 2–4 hrs availability)	Client PM	Day 1
6	Existing test cases, defect data, release cadence data	Client QA lead	Day 2
7	Security/procurement pre-clearance for tool trials & AI/cloud tools (data residency)	Client Security	Day 3

#	Item	Owner	Needed by
8	Sample test data or data-generation permission	Client QA	Day 4

1.3 Stakeholders & RACI

Activity	You (Architect)	Client QA Lead	App SMEs	Client IT	Sponsor
App assessment	R/A	C	C	I	I
Scenario shortlisting	R/A	C	C	I	I
Tool shortlisting	R/A	C	I	I	A (sign-off)
Pilot build	R/A	I	C	C	I
Recommendation & roadmap	R/A	C	I	I	A (sign-off)

1.4 Cadence

- Daily 15-min standup with client QA lead.
- Weekly 45-min checkpoint with sponsor (end of each week; demo-driven).
- Risk log reviewed every Friday.

---

2. Week-by-Week Plan (Day-Level)

Week 1 — Discovery & Assessment (all 9 apps)

Day	Activities	Output
1	Kickoff (60 min); confirm scope, success criteria, access; distribute app intake questionnaire (§3.1)	Signed-off PoA charter, success criteria
2	Assess Apps 1–3: SME walkthrough (45 min each) + technical probe (§3.3)	Assessment worksheets 1–3
3	Assess Apps 4–6 (same pattern)	Assessment worksheets 4–6

Day	Activities	Output
4	Assess Apps 7–9; mine defect/test-case data for hotspots	Assessment worksheets 7–9
5	Score all apps (§3.2), classify Green/Amber/Red, draft candidate scenario list; Week-1 checkpoint	Draft Assessment Report v0.5, candidate scenario backlog (20–30 items)

Week 2 — Scenario & Tool Shortlisting + Environment Setup

Day	Activities	Output
6	Score scenarios (§4.2); shortlist 8–12 (must include 2–3 "hard" desktop cases)	Shortlisted scenario matrix, client sign-off
7	Tool long-list → apply knockout criteria (§5.2); build weighted scorecard	Tool scorecard (paper evaluation complete)
8	Select 2 pilot tools; validate licenses; install & configure on client VM	2 tools operational
9	Smoke-test each tool against the hardest app (object recognition spike, §3.3); build skeleton framework (folder structure, config, reporting)	Spike results, framework skeleton per tool
10	Prepare test data, baseline manual execution times for shortlisted scenarios; Week-2 checkpoint	Baseline metrics sheet, environment sign-off

Week 3 — Pilot Build (the core sprint)

Day	Activities	Output
11–12	Automate scenarios 1–4 in Tool A and Tool B in parallel (same scenarios in both — this is essential for comparison)	4 scenarios × 2 tools
13–14	Automate scenarios 5–8 (include the hard desktop cases); log every metric per §6.3 as you go	8 scenarios × 2 tools
15	Stability runs: execute full suite 10× per tool (mix of environments/time-of-day); trigger one deliberate app UI change and measure maintenance effort; Week-3 checkpoint demo	Flakiness & maintenance data, execution videos

Week 4 — Analysis, Recommendation & Roadmap

Day	Activities	Output
16	Consolidate pilot metrics; compute ROI model (§7.3); finalize tool scorecard with empirical scores	Pilot Results Report v1
17	Author AI-native testing approach (§8) tailored to chosen tool	AI-Native Testing Approach doc
18	Finalize Assessment Report; build recommendation deck; draft roadmap (§9.5)	Final report + deck v0.9
19	Internal dry-run with QA lead; incorporate feedback; package scripts + README + handover notes	Deliverable package v1.0
20	Executive playback (60–90 min): live demo, recommendation, roadmap, Q&A; obtain sign-off	Signed-off PoA, next-phase agreement

Timeline protection rules: timebox any single automation blocker to 4 hours, then switch to a fallback approach (e.g., image-based recognition) and log it as a finding — a documented limitation is itself valuable PoA evidence. Never let one app consume the sprint.

---

3. Application Assessment Framework

3.1 Intake Questionnaire (per app — send Day 1, review in SME session)

1. App type & tech stack (WinForms, WPF, Java Swing/SWT, Delphi, Electron, Qt, mainframe emulator, Citrix-published, web framework, shell/CLI)?
2. UI framework version and any custom/third-party control libraries (DevExpress, Telerik, Infragistics, SAP GUI…)?
3. Deployment model: local install, Citrix/RDP, VDI, browser?
4. Release cadence & typical change volume per release?
5. Current regression suite size, manual execution effort (person-days/cycle)?
6. Existing automation (tool, coverage %, health)? Why did it fail/succeed?
7. Top 5 defect-prone areas (from defect data, not opinion)?
8. Test data: how created, refreshed, masked? Any golden datasets?
9. Environment availability and stability (uptime, refresh cycles, contention)?
10. Integrations/upstream-downstream dependencies used in E2E flows?

11. Authentication (SSO, MFA, smartcard — MFA is a common automation blocker)?
12. Compliance constraints (data can/cannot leave premises — determines cloud/AI tool eligibility)?

3.2 Assessment Scoring Model (score each app 1–5 on each dimension)

Dimension	Weight	5 (best)	1 (worst)
Object identifiability (technical testability)	20%	Standard controls, stable automation IDs exposed via UIA/DOM	Custom-drawn controls, no accessibility tree, image-only
App & environment stability	15%	Stable env, predictable behavior	Frequent crashes, env contention
Test data readiness	10%	Golden data, self-service refresh	Manual, scarce, prod-only data
Change frequency vs. locator stability	10%	Stable UI between releases	UI churns every release
Business criticality of regression	15%	Revenue/compliance critical flows	Low-impact internal utility
Manual regression effort (automation payoff)	15%	>10 person-days per cycle	<0.5 person-day
Existing automation reusability	5%	Healthy assets to extend	None or abandoned
Workflow complexity (multi-app, OCR, files, DB)	10%	Simple single-app flows	Heavy cross-app + legacy interop

Automation Feasibility Index (AFI) = Σ(score × weight) → normalize to 100.

- **Green (≥70):** automate early, high ROI.
- **Amber (45–69):** automate with mitigations (test data fixes, ID injection, image fallback).
- **Red (<45):** defer, or use as PoA "stress case" to differentiate tools.

Present both an **AFI ranking table** and a **2×2 plot** (Feasibility vs. Business Value) — the 2×2 is what executives remember.

3.3 Desktop Technical Probe (hands-on, 30–45 min per desktop app — do not skip)

1. Run **Accessibility Insights for Windows** / **inspect.exe** against key screens → does the UI Automation (UIA) tree expose names, AutomationIds, control types? Screenshot the tree for the report.
2. Try **FlaUI/UIA snippet or the candidate tool's spy** on the 3 most complex controls (grids, tree views, custom widgets). Record: recognized natively / recognized partially / image-only.
3. Check for blockers: elevated (admin) windows, Citrix/RDP layer (pixels only), embedded browser panels (CEF/WebView2), owner-drawn controls, modal native dialogs, MFA prompts.
4. Note remediation levers: can dev team inject AutomationIds? Is a Citrix-side agent permissible? Is API/DB seam available to bypass UI for setup steps?
5. For shell scripts: identify interface (exit codes, stdout/stderr, files, DB writes) — these are usually best automated with pytest/bats + assertions, not a UI tool; capture this as a finding.

3.4 Per-App Assessment Worksheet Template

```
App Name / ID:                    Owner / SME:
Type & tech stack:                Deployment model:
UIA/DOM exposure summary:         (attach inspect.exe screenshot)
Complex controls & recognition result:
Blockers found:                   Remediation levers:
Regression suite size / manual effort per cycle:
Defect hotspots:                  Test data status:
Dimension scores (8 rows):        AFI score:      RAG:
Candidate scenarios from this app:
Notes / risks:
```

---

4. Test Scenario Shortlisting

4.1 Sourcing (don't rely only on client-flagged pain points)

- Defect density analysis (top modules by defect count, last 6–12 months).
- Highest manual-effort regression packs.
- Cross-application E2E flows (these expose tool orchestration ability).

- Deliberately include **technical challenge patterns**: custom grid manipulation, dynamic object IDs, embedded browser inside desktop app, OCR/image validation, file export + content verification, DB validation step, multi-window/modal handling, Citrix or remote session, long-running batch triggered by shell script, data-driven iteration.

4.2 Scenario Scoring Matrix (score 1–5 each; shortlist top 8–12)

Criterion	Weight
Business criticality of the flow	25%
Execution frequency (per release/cycle)	15%
Manual effort per execution	15%
Defect history of the area	10%
Technical challenge coverage (differentiates tools)	25%
Feasibility within PoA timeline	10%

Balance rule: ~50% high-value/representative flows, ~30% technically hard cases, ~20% quick wins (to demo velocity). Cover at least: 2 desktop apps (incl. the hardest), 1 web app, 1 shell/CLI, 1 cross-app E2E.

4.3 Shortlist Register Template

ID	Scenario	App(s)	Type	Challenge pattern(s)	Manual effort	Priority score	Rationale

---

5. Tool Evaluation & Shortlisting

5.1 Candidate Long List (by capability area)

- **Enterprise commercial (strong desktop):** Tricentis Tosca, OpenText UFT One, SmartBear TestComplete, Ranorex, Eggplant (image/AI-driven, good for Citrix), UiPath Test Suite (strong on legacy/Citrix via RPA heritage).
- **Open source desktop:** WinAppDriver + Appium, FlaUI, Pywinauto, White/TestStack, AutoIt (fallback), SikuliX (image fallback).

- **Web:** Playwright, Selenium, Cypress (often paired with a commercial desktop tool for hybrid stacks).
- **AI-native / AI-augmented:** testRigor, Testim/Tricentis, mabl, Functionize, Applitools (visual AI), AskUI (vision-based desktop), Katalon AI features; plus agentic/LLM-based generation (e.g., Playwright + MCP/LLM agents) for the AI-native approach section.
- **Shell/CLI & API layer:** pytest/bats, REST Assured/Postman/Karate — recommend as complementary layer regardless of UI tool choice.

Verify current capabilities, licensing and AI features with vendors during Week 2 — this market moves quickly.

5.2 Knockout Criteria (apply first — anything failing these leaves the list)

1. Cannot recognize the client's dominant desktop technology natively or via viable fallback.
2. Licensing/procurement impossible within trial window or budget class.
3. Violates client security/data-residency policy (esp. cloud-only AI tools).
4. No CI/CD integration path.
5. Vendor viability/support concerns for enterprise use.

5.3 Weighted Tool Scorecard (paper eval Week 2 → empirical scores Week 4)

Criterion	Weight	Tool A	Tool B
Desktop technology support (client's actual stacks)	20%		
Object recognition robustness (measured in pilot)	15%		
AI capabilities (self-healing, NLP authoring, visual AI, failure triage)	12%		
Script maintainability & reusability (framework model, versioning)	10%		
CI/CD & DevOps integration (Jenkins/Azure DevOps/GitLab, containerized/remote execution)	8%		
Skills availability & learning curve (client team profile)	8%		
Licensing model & 3-yr TCO	8%		
Cross-platform coverage (web, API, mobile future-proofing)	6%		
Reporting & analytics (dashboards, traceability to ALM/Jira)	5%		

Criterion	Weight	Tool A	Tool B
Enterprise support, community, roadmap	5%		
Test data & environment management features	3%		
Rule: where possible, replace paper scores with pilot-measured evidence (recognition success rate, flakiness, maintenance minutes) before final scoring.			

6. Pilot Execution Standards

6.1 Framework Skeleton (per tool — keep identical shape for fair comparison)

```
/framework
  /config          (env URLs, app paths, timeouts, credentials via vault)
  /object-repo     (or page/screen objects)
  /tests           (one scenario per file, tagged by app & pattern)
  /utils           (data helpers, DB/file validators, OCR helper)
  /reports         (HTML + machine-readable results)
  /ci              (pipeline YAML — prove at least one CI run)
  README.md        (setup, run commands, known limitations)
```

6.2 Build Rules

- Same engineer effort budget per tool per scenario (log actual hours).
- No heroics: 4-hour blocker timebox rule (see §2).
- Every run logged; every failure classified: *app defect / environment / test data / object recognition / tool defect / script defect.*
- One deliberate UI change on Day 15 → measure minutes-to-repair in each tool (maintenance evidence).

6.3 Pilot Metrics to Capture (per scenario × per tool)

Metric	Definition	Target/Use
Script development time (hrs)	First green run, incl. object mapping	Compare authoring velocity

Metric	Definition	Target/Use
Object recognition success rate	% of UI elements identified natively (no image fallback)	Core desktop differentiator
Execution time vs. manual	Automated runtime ÷ manual baseline	ROI input
Pass rate over 10 stability runs	Passes ÷ runs on unchanged app	≥95% expected
Flakiness rate	% runs failing for non-defect reasons	<5% target
Self-healing events	# auto-repaired locators (AI tools)	Evidence for AI value
Maintenance effort	Minutes to repair after deliberate UI change	Lower = better TCO
Reusability	% of shared components reused across scenarios	Framework quality
CI execution proof	Ran headless/agent-based in pipeline? Y/N + duration	Enterprise readiness
Defects found by automation	Real defects surfaced during pilot	Instant value story

7. Metrics Catalog & ROI Model

7.1 Assessment-Phase Metrics

AFI per app; % apps Green/Amber/Red; total manual regression effort across portfolio (person-days/cycle); defect density by module; % of UI elements exposing automation IDs (per desktop app).

7.2 Pilot-Phase Metrics — §6.3 table.

7.3 Business-Case / ROI Model

- **Annual manual cost** = cycles/yr × person-days/cycle × loaded day rate.
- **Automation cost (yr 1)** = licenses + infra + (scenarios × avg build hrs × rate) + maintenance (assume 15–25% of build effort/yr, use your measured maintenance data to justify).
- **Break-even runs per script** = build cost ÷ (manual cost per run − automated cost per run).

- **Projected coverage curve**: scenarios automated per sprint at pilot-measured velocity → 6/12-month coverage %.
- **Cycle-time compression**: regression duration before vs. projected after (days → hours).
- Present a 3-year TCO comparison per tool: license + infra + build + maintenance + training.

7.4 Scaled-Program KPIs (for the roadmap)

Automation coverage % of regression; automated pass rate; flaky-test %; mean time to repair a broken script; escaped defects trend; regression cycle duration; % test runs in CI vs. manual trigger.

---

8. AI-Native Testing Approach (deliverable structure)

Frame AI along four layers, mapped to concrete tool capabilities and guardrails:

1. **Authoring:** NLP/plain-English test creation; LLM-assisted script generation from requirements/user stories; model-based generation from recorded flows. *Metric: authoring time reduction vs. Week-3 baseline.*
2. **Execution resilience:** self-healing locators (multi-attribute + visual matching); visual AI validation (Applitools-style) for pixel-rendered/Citrix apps; smart waits. *Metric: self-healing events, flakiness reduction.*
3. **Analysis:** AI failure triage/clustering (app defect vs. script vs. env), auto root-cause hints, risk-based test selection (run only tests impacted by a change). *Metric: triage time per failed run.*
4. **Autonomy (forward-looking):** agentic exploratory testing on new builds; LLM agents driving the app (e.g., vision-based agents for legacy desktop where no object model exists) — position as Phase 3 innovation track with a controlled experiment, not a Day-1 dependency.

Guardrails to state explicitly: human review of AI-generated tests before merge; data-privacy review for any cloud AI (masking, on-prem options); measure AI value with the same metrics (don't adopt on vendor claims); version-control everything AI generates.

---

9. Deliverable Templates

9.1 Application Assessment Report (15–25 pages)

1. Executive summary (1 page: portfolio RAG map, headline numbers, top 5 findings)

2. Assessment approach & scoring model
3. Portfolio view: AFI ranking table + Feasibility-vs-Value 2×2
4. Per-app profiles (1–1.5 pages each, from §3.4 worksheets, with UIA evidence screenshots)
5. Cross-cutting findings (test data, environments, MFA, Citrix, skills)
6. Remediation recommendations per Amber/Red app
7. Shortlisted scenario register (§4.3)
8. Appendix: raw scores, questionnaire responses

9.2 Tool Evaluation Scorecard — §5.3 matrix + one page of narrative per tool (strengths, limitations observed, licensing summary, fit statement).

9.3 Pilot Results Report

1. Scope: scenarios × tools matrix, effort spent
2. Metrics dashboard (§6.3, side-by-side per tool)
3. Evidence: screenshots/video links, CI run links, defects found
4. Findings per challenge pattern (e.g., "custom grid: Tool A native, Tool B image-fallback — 3× maintenance cost")
5. Limitations & deviations (timeboxed blockers)
6. Conclusion feeding final scorecard

9.4 Final Recommendation Deck (executive, 12–18 slides)

Context & objectives → approach → portfolio assessment highlights → pilot evidence (demo/video) → scorecard & recommendation (primary tool + complementary stack: web/API/shell layers) → AI-native approach summary → ROI & TCO → roadmap → risks & asks → next steps.

9.5 Scaled Automation Roadmap

Phase	Timeline	Focus	Exit criteria
Phase 0 — Foundation	Weeks 1–6 post-PoA	Procurement, framework hardening, CI/CD wiring, test data strategy, team enablement plan	Tool live in CI, 2 engineers trained, coding standards published
Phase 1 — First wave	Months 2–4	Automate Green apps' top regression packs (target 30–40% coverage of priority regression)	Coverage & flakiness KPIs met, regression cycle reduced ≥X%

Phase	Timeline	Focus	Exit criteria
Phase 2 — Scale	Months 4–8	Amber apps with remediations; cross-app E2E; nightly CI regression; dashboarding	60–70% priority coverage; MTTR for scripts < target
Phase 3 — Optimize & AI	Months 8–12	Risk-based selection, self-healing at scale, agentic exploration pilot, shift-left (in-sprint automation)	In-sprint automation ⩾80% of new stories; escaped defects trending down
Include per phase: team model (CoE vs. federated), skills/training, governance (definition of done for automated tests, review gates), and budget line items.			

10. Risk Register (pre-populated for a 4-week PoA)

Risk	Likelihood	Impact	Mitigation
Access/licenses delayed	High	High	Entry-criteria gate before Day 1; escalate Day 2
Hard desktop app blocks pilot	Med	High	4-hr timebox rule; image fallback; report as finding
SME unavailability	Med	Med	Book all sessions in Week-1 kickoff; record walkthroughs
Env instability skews metrics	Med	Med	Classify failures by cause; rerun stability window
Security blocks cloud AI tools	Med	Med	Confirm Day 3; keep one on-prem-capable tool in shortlist
Scope creep (client adds apps/scenarios)	High	Med	Change control via charter; park in roadmap backlog

11. Exit Checklist (Definition of Done)

- [ ] 9/9 apps assessed, scored, RAG-classified with evidence
- [ ] 8–12 scenarios shortlisted and signed off
- [ ] Same scenario set automated in both tools; 10 stability runs each; ≥1 CI execution proven
- [ ] All §6.3 metrics captured; maintenance experiment done
- [ ] All five deliverables issued v1.0; scripts + README handed over in client repo
- [ ] Executive playback held; recommendation and next-phase decision recorded